

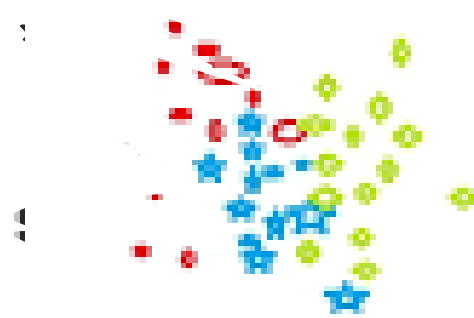


fig 1: before applying
k-means clustering

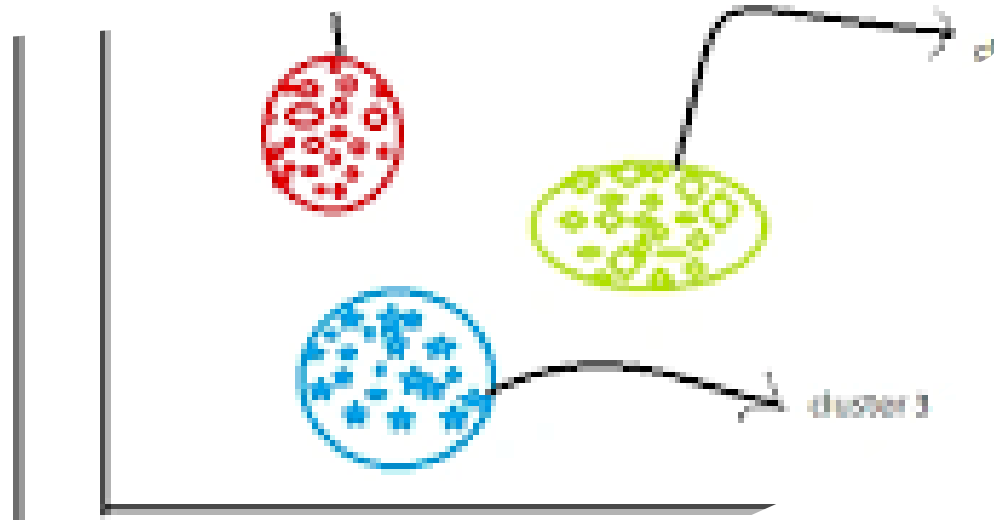
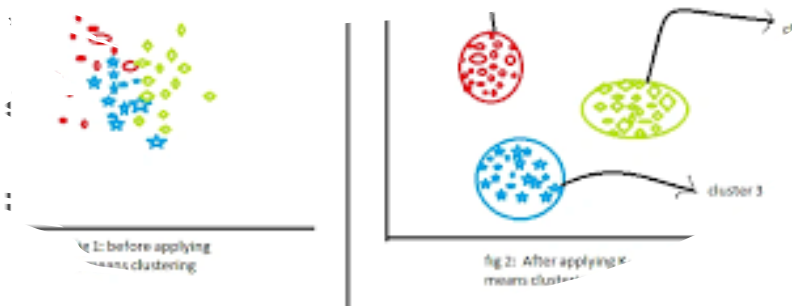


fig 2: After applying k-
means clustering

Clustering

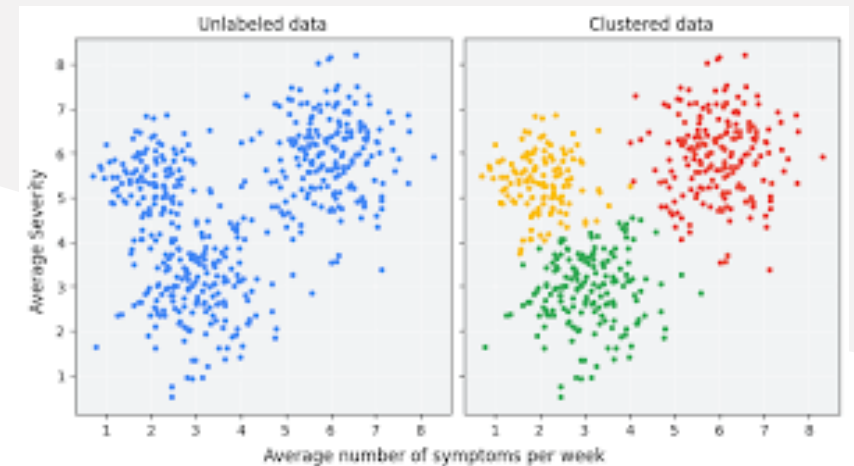
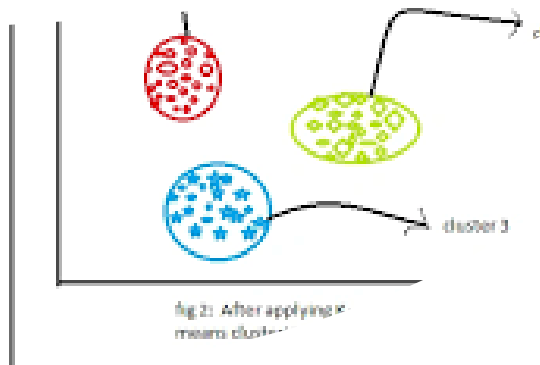
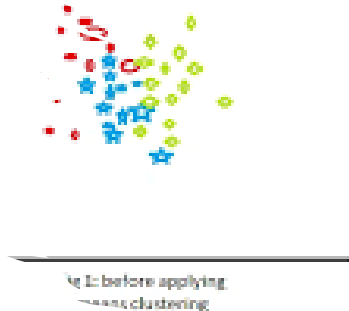
¿Qué es y para qué sirve el clustering?

La agrupación en clústeres es un algoritmo de machine learning no supervisado que organiza y clasifica diferentes objetos, puntos de datos u observaciones en grupos o clusters basados en similitudes o patrones.

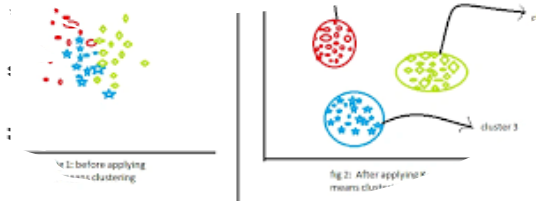
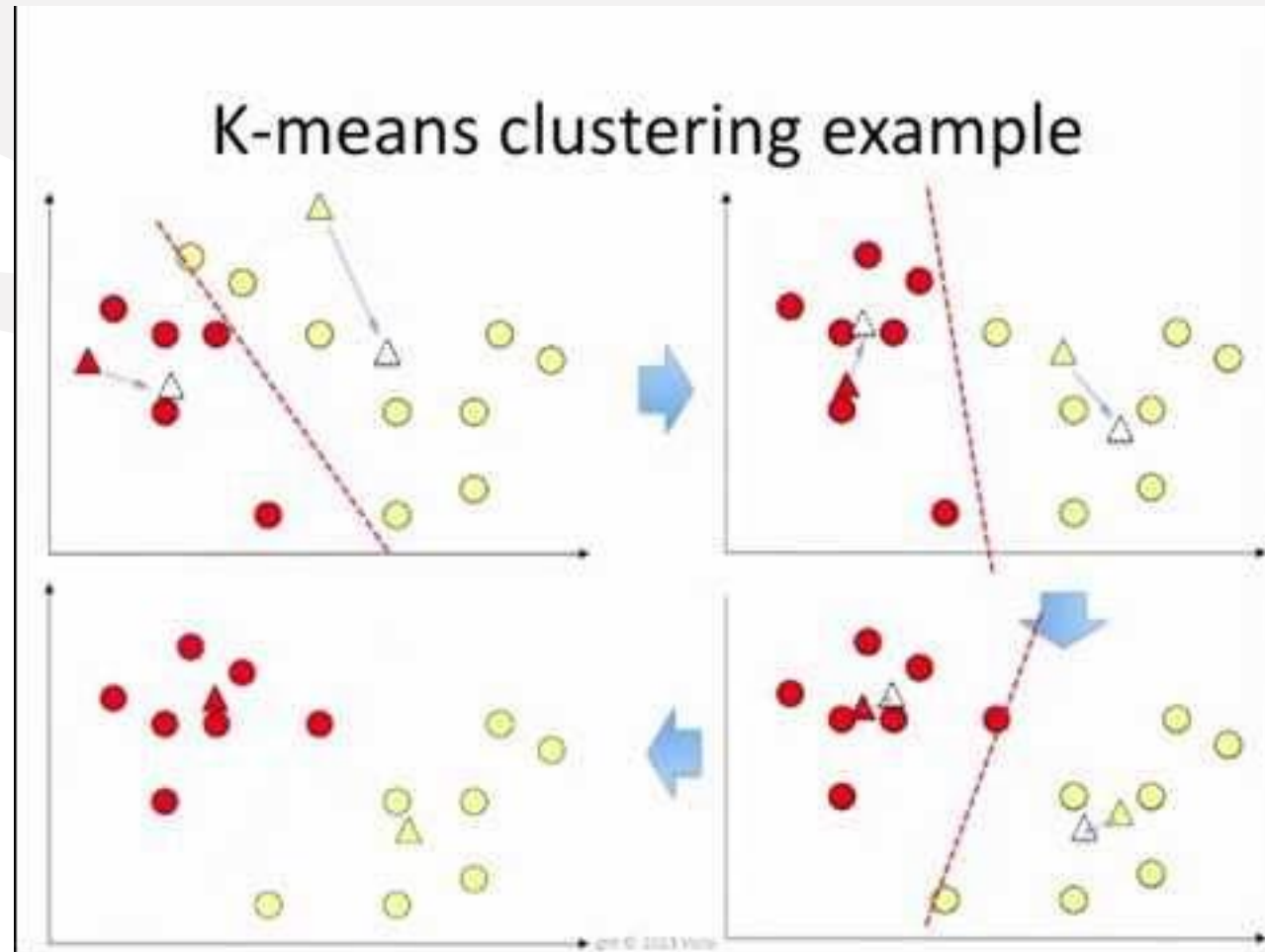


¿Qué es y para qué sirve el clustering?

Su objetivo es agrupar un conjunto de datos en **grupos o clústeres que compartan características similares** entre sí, pero sean diferentes de los grupos vecinos.

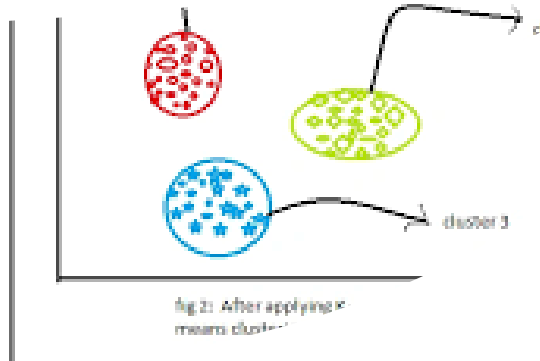
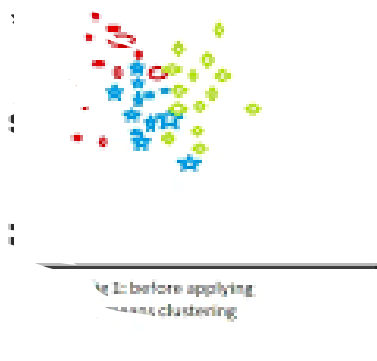


¿Qué es y para qué sirve el clustering?



¿Qué es y para qué sirve el clustering?

La clusterización es la categorización de la información del consumidor para generar segmentaciones relevantes que nos permitan crear campañas publicitarias personalizadas más efectivas. Son grupos de personas con **características similares**: gustos, necesidades o motivaciones.

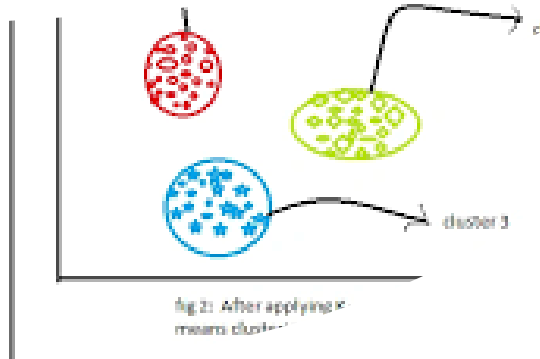
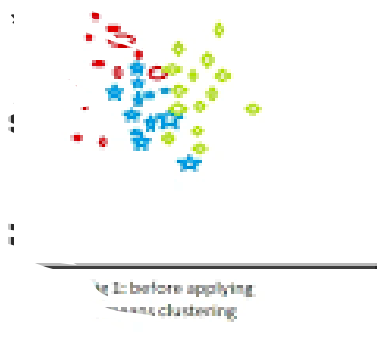


¿Qué es y para qué sirve el clustering?

Ejemplos típicos:

- Segmentar clientes (por hábitos de compra)
- Agrupar noticias por tema sin haberlas etiquetado
- Encontrar “tipos” de sensores / máquinas por patrones de funcionamiento

Detectar “cosas raras” (outliers) como puntos que no encajan en ningún grupo

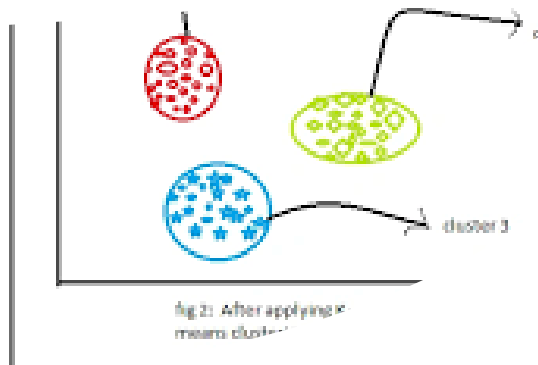
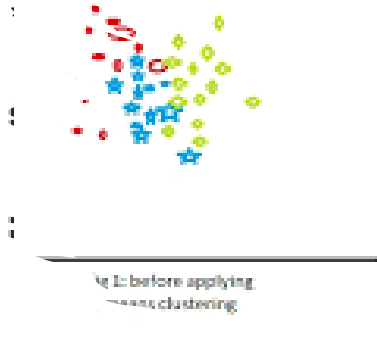


¿Qué es y para qué sirve el clustering?

¿Qué significan esas características similares?

Depende de cómo midas la distancia o similitud entre puntos:

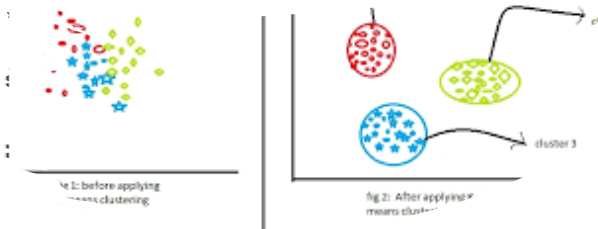
- En datos numéricos se usa mucho distancia euclídea (la de “regla” en un plano)
- En texto se usan similitudes basadas en vectores (embeddings), coseno, etc.



¿Qué es y para qué sirve el clustering?

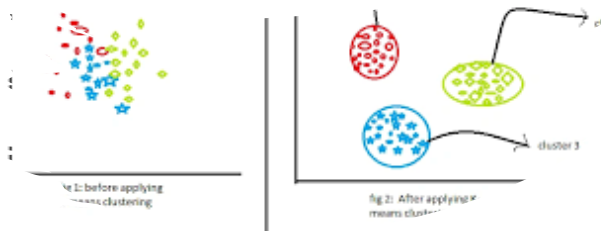
Flujo típico de trabajo (paso a paso)

1. Seleccionar variables (features): ¿qué columnas describen bien el problema?
2. Escalar/normalizar (muy importante si hay variables con magnitudes distintas)
3. Elegir un algoritmo de clustering
4. Decidir el número de clusters (si el método lo requiere)
5. Evaluar e interpretar los grupos: ¿tienen sentido? ¿cómo los describo?



K-Means

El método de K-means clustering es un método no jerárquico para agrupar objetos (no variables) que particiona el set de datos en K clústeres distintos y no solapantes, lo que significa que ninguna observación puede pertenecer a más de un clúster. El número de clústeres o subgrupos requeridos se ha de establecer al inicio (con lo que es importante tener un buen conocimiento de los datos).

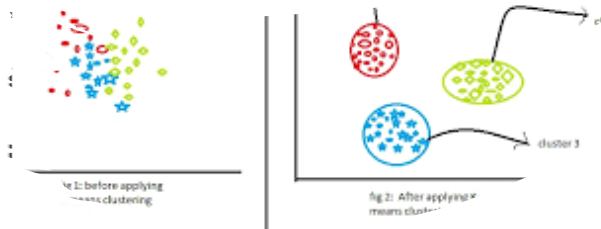


K-Means

- Muy popular por ser simple y rápido
- Requiere elegir K = número de grupos
- Funciona bien cuando los clusters son “más o menos redondos” y de tamaño similar

Sensible a:

- Escalado
- Outliers
- Elección de K
- Intuición: el algoritmo busca “centros” (centroides) y asigna cada punto al centro más cercano.

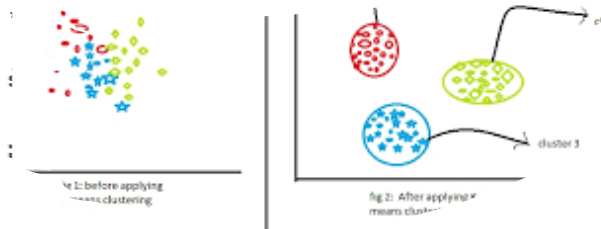


K-Means

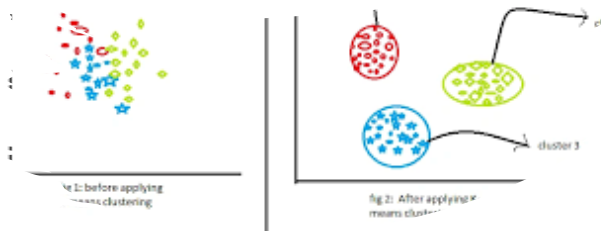
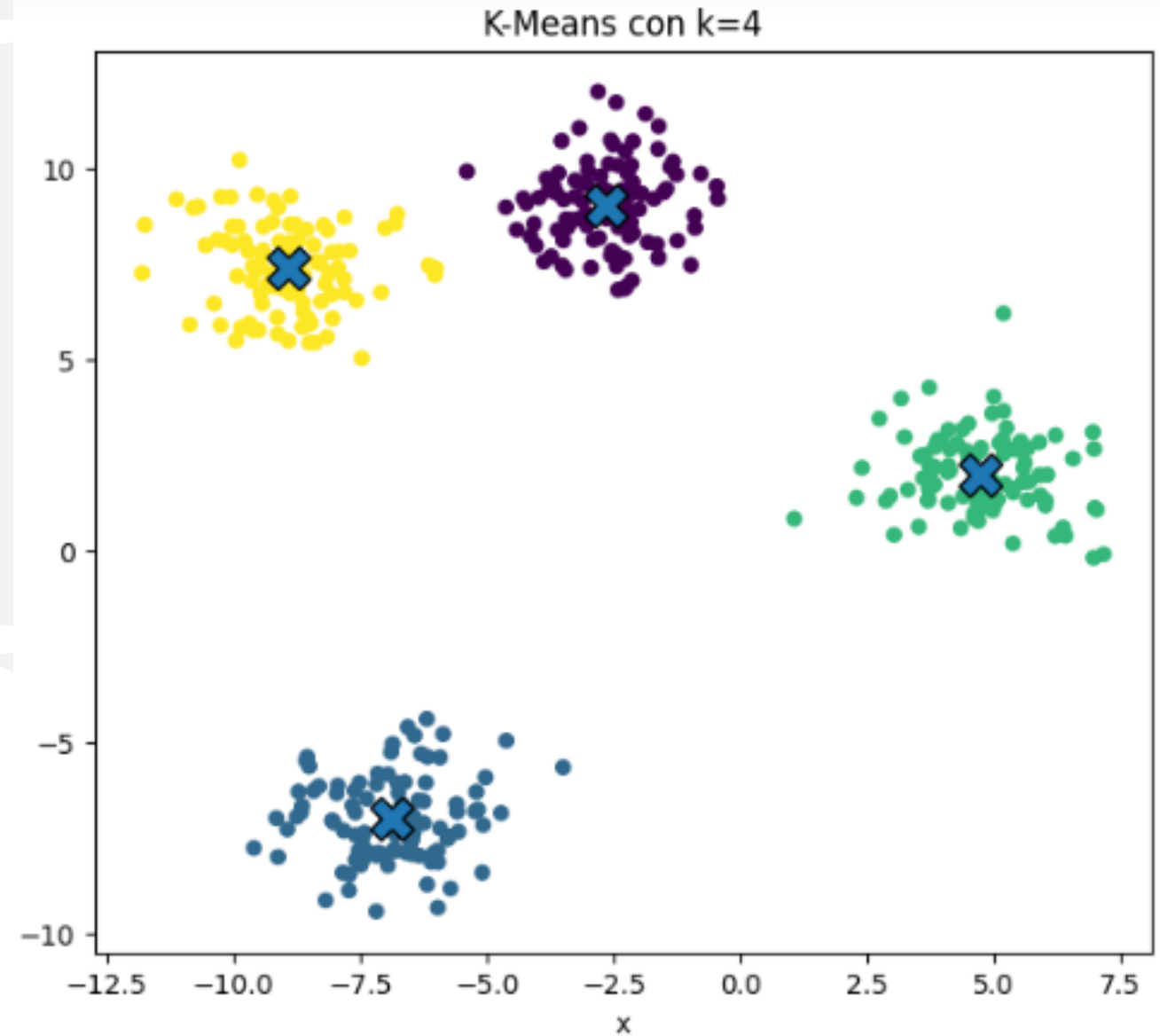
¿Cómo elegir el número de clusters?

Si el método lo necesita (como K-Means), dos ideas típicas:

- Método del codo (Elbow): buscar el punto donde mejorar deja de ser “gran cosa”
- Silhouette score: mide si un punto está bien dentro de su cluster y lejos de otros
- Aun así, no hay un “K verdadero” siempre: a veces depende del objetivo del negocio.



K-Means

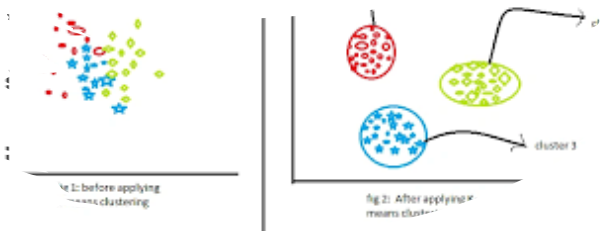


K-Means

¿Cómo se evalúa si “está bien”?

En clustering no existe “accuracy” porque no hay etiquetas. Se combina:

- Métricas internas: silhouette, cohesión/separación
- Visualización: PCA/t-SNE/UMAP para “ver” en 2D (con cuidado)
- Interpretación: describir cada grupo (medias, medianas, variables clave)
- Utilidad: ¿sirve para tomar decisiones? (marketing, riesgos, operaciones...)

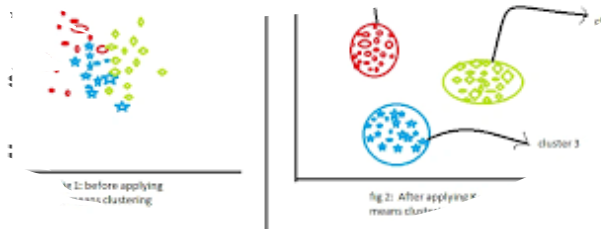


K-Means

¿Cómo elegir el número de clusters?

Si el método lo necesita (como K-Means), dos ideas típicas:

- Método del codo (Elbow): buscar el punto donde mejorar deja de ser “gran cosa”
- Silhouette score: mide si un punto está bien dentro de su cluster y lejos de otros
- Aun así, no hay un “K verdadero” siempre: a veces depende del objetivo del negocio.

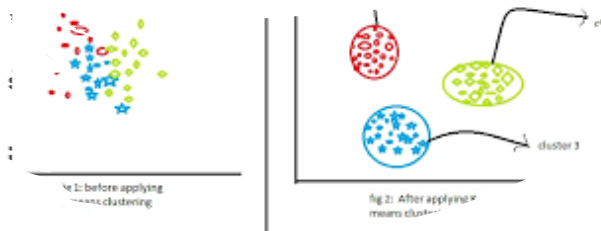


K-Means

Proceso de iteración

En K-Means, en cada iteración pasa esto:

1. Asignación: para cada cliente, calculas la distancia a cada centroide (los “centros” de los clusters) y lo asignas al cluster del centroide más cercano.
2. Actualización: para cada cluster, recalculas su centroide como la media de los clientes que han caído en ese cluster.
3. Repetir: vuelves a hacer 1) y 2) hasta que ya no cambian las asignaciones (o los centroides se mueven muy poco).

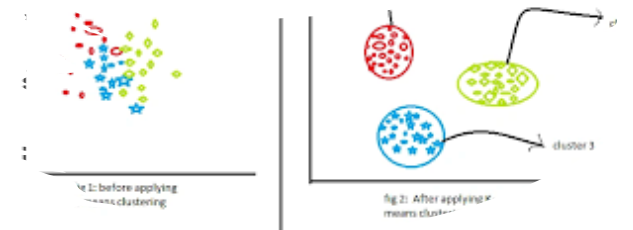


Método del codo con K-Means

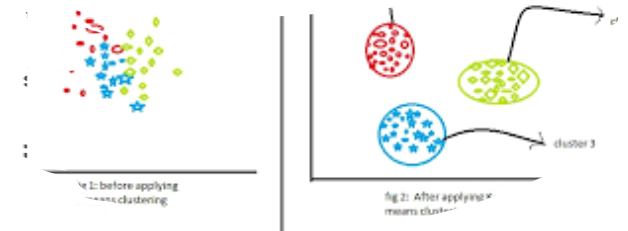
Elegir k mirando cómo baja la “inercia” al aumentar el número de clusters.

- Generamos un dataset de ejemplo con `make_blobs` (datos artificiales en 2D).
- Para cada k (1..10), entrenamos K-Means y guardamos `km.inertia_`.
- Dibujamos inercia vs k y buscamos el punto donde la curva se aplanan (el “codo”).

Nota: el codo es una heurística (no siempre hay un codo clarísimo).



Calculamos la inercia para distintos k y la representamos.

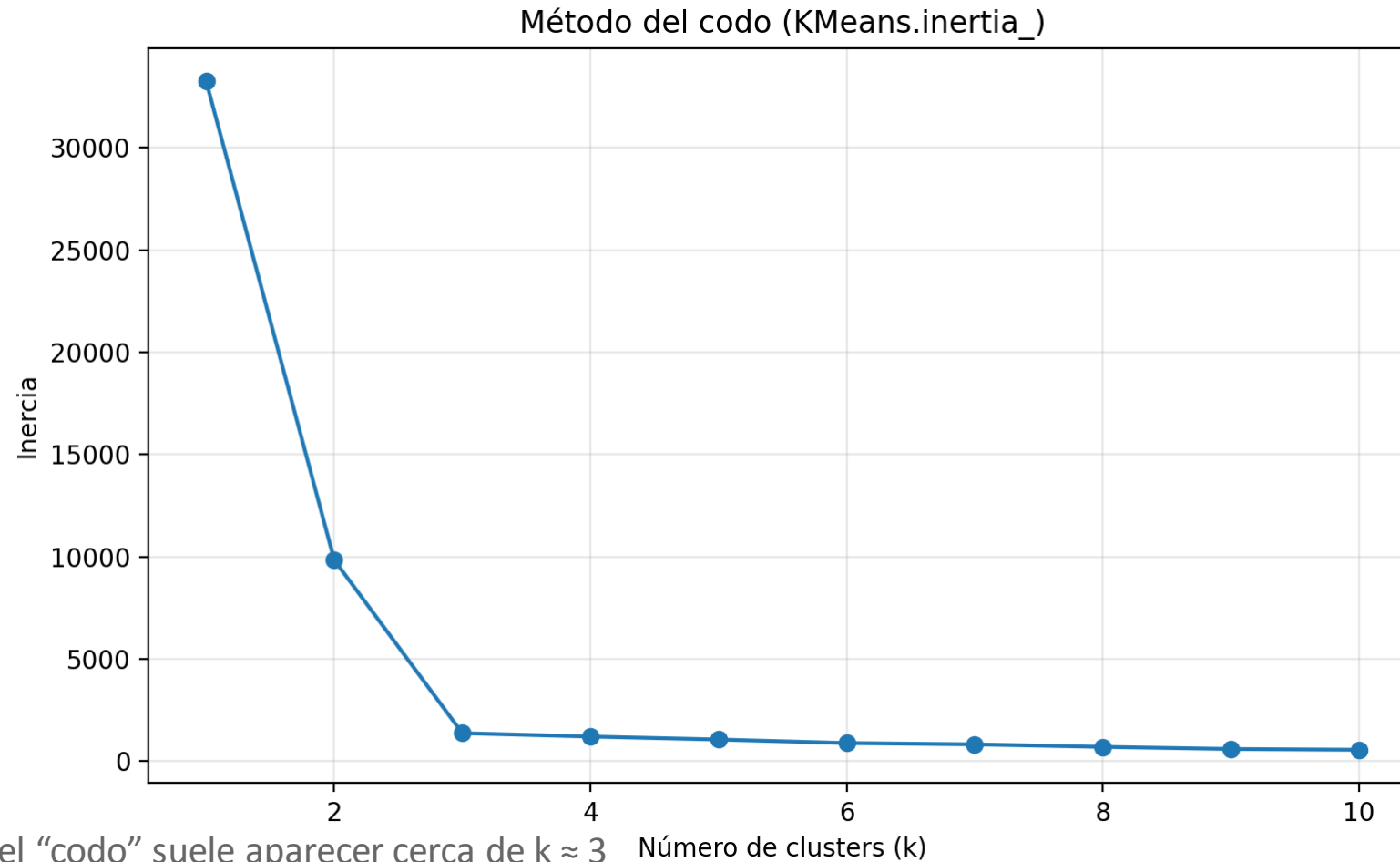
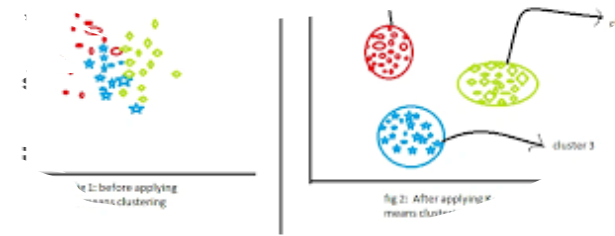


```
1 import matplotlib.pyplot as plt
2 from sklearn.cluster import KMeans
3 from sklearn.datasets import make_blobs
4
5 # Dataset de ejemplo (3 nubes)
6 X, _ = make_blobs(n_samples=500, centers=3, cluster_std=1.2, random_state=42)
7
8 ks = range(1, 11)
9 inertias = []
10
11 for k in ks:
12     km = KMeans(n_clusters=k, random_state=42, n_init="auto")
13     km.fit(X)
14     inertias.append(km.inertia_) # <- inercia = within-cluster sum of squares
15
16 plt.figure(figsize=(7, 5))
17 plt.plot(list(ks), inertias, marker="o")
18 plt.title("Método del codo (KMeans.inertia_)")
19 plt.xlabel("Número de clusters (k)")
20 plt.ylabel("Inercia")
21 plt.grid(True, alpha=0.3)
22 plt.show()
```

Tip: si cambias centers=3 por centers=5, el "codo" suele moverse cerca de 5.

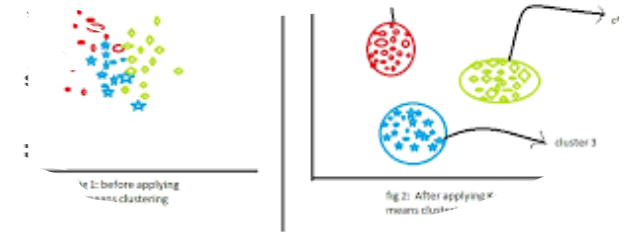
Curva del codo: inercia vs k

La inercia baja rápido al principio y luego mejora cada vez menos.



En este ejemplo, el “codo” suele aparecer cerca de $k \approx 3$ porque el dataset se generó con centers=3.

Cómo elegir k (muy práctico)



- 1) Calcula la inercia para $k = 1..10$ (o más).
- 2) Dibuja la curva y busca el punto donde deja de bajar “a lo bestia”.
- 3) Prueba 2–3 valores alrededor del codo y mira si los grupos tienen sentido.
- 4) Si dudas: complementa con silhouette score y con interpretación del negocio.

Regla de oro: si subir k solo mejora “un poquito” pero complica mucho la explicación, quédate con un k más simple.

Ejemplo: K-Means

Objetivo: agrupar puntos y visualizar clusters + centroides

¿Qué hace make_blobs(...)?

- Crea datos “de juguete” con nubes (clusters) en 2D
- n_samples: nº de puntos totales
- centers: nº de grupos “reales”
- cluster_std: dispersión (más grande = más mezcla)
- random_state: reproduce siempre lo mismo

K-Means en 4 ideas

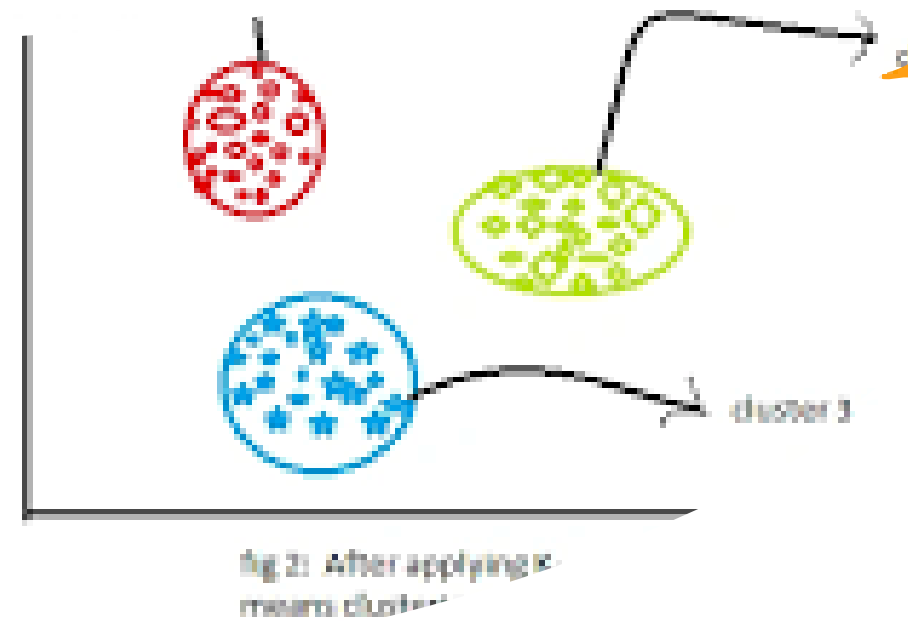
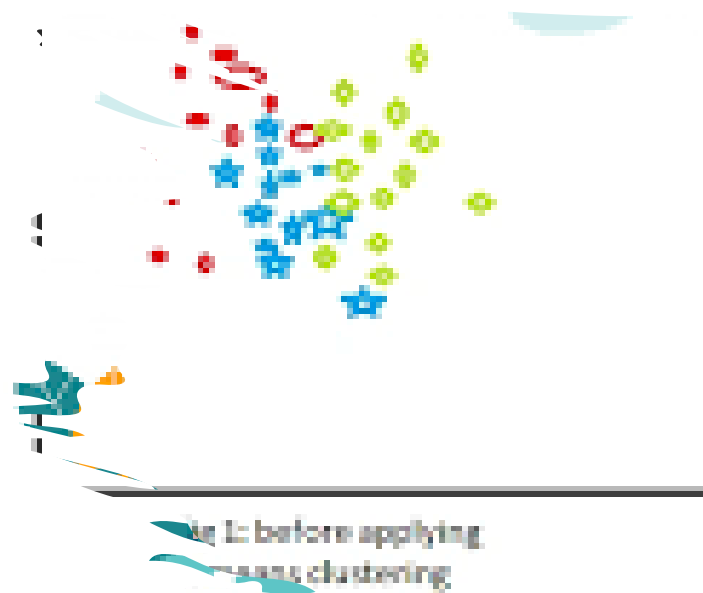
- 1) Elige k (nº de clusters)
- 2) Ajusta: km = KMeans(n_clusters=k, ...)
- 3) Etiquetas: labels = km.fit_predict(X)
- 4) Centros: km.cluster_centers_

Tips rápidos:

- Escala variables si no están en la misma unidad
- Prueba varios k (codo / silhouette)
- Los outliers pueden “tirar” de los centroides

Código mínimo (copiar/pegar)

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
X, _ = make_blobs(n_samples=400, centers=4, cluster_std=1.1,
random_state=42)
k = 4
km = KMeans(n_clusters=k, random_state=42, n_init=10)
labels = km.fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=labels, s=25)
plt.scatter(km.cluster_centers_[:, 0], km.cluster_centers_[:, 1],
marker="X", s=250, edgecolors="k")
plt.title(f"K-Means (k={k})")
plt.show()
```



Gracias